



Universidad Argentina de la Empresa
Facultad de Ingeniería y Ciencias Exactas

Proceso de Desarrollo Software TPO

Profesora: Leon María Angela

Equipo: "Proyecto Y"

Integrantes:

Perez Leal, Agustín - 1189636

Martinez Zorzi, Facundo - 1193730

Mioni Bonuccelli, Franco Brunello - 1189642

Smith, Pedro - 1195404

1. Diagrama de clases UML

Adjunto en entrega

2. Diagramas de Secuencia

3.1 CU02 — Simular Partido

Adjunto en entrega

3.2 CU01 — Gestionar Equipo y Jugadores

Adjunto en entrega

3. Patrones de Diseño Aplicados

Patrón	Tipo	Clases principales
Builder	Creacional	IBuilder, PartidoBuilder, Partido
Facade	Estructural	ControladorPartido
Strategy	Comportamiento	ITactica, TacticaOfensiva, TacticaDefensiva, TacticaEquilibrada
Observer	Comportamiento	IObservadorPartido, Marcador, Estadisticas, RelatoDeportivo
State	Comportamiento	IEstadoPartido, EstadoPrimerTiempo, EstadoEntreTiempo, EstadoSegundoTiempo, EstadoFinalizado

4. Justificación de cada Patrón

5.1 Builder — Construcción de Partido

Problema: crear un Partido implica configurar dos equipos, asignar tácticas, registrar observadores y conectar el motor de simulación. Hacerlo todo en el constructor resulta en un constructor con demasiados parámetros y lógica de inicialización mezclada.

Solución: PartidoBuilder encapsula ese proceso paso a paso. El cliente llama setEquipoLocal(), setEquipoVisitante(), setTactica() y build(), sin necesidad de conocer el orden ni los detalles internos.

Justificación concreta: sin este patrón, crear partidos con distintas configuraciones (por ejemplo, en un torneo) requeriría repetir toda la lógica de armado cada vez.

5.2 Facade — Punto de entrada al sistema

Problema: la UI necesita coordinar el builder, el motor de simulación, el marcador y la persistencia. Si la UI accede directamente a cada componente, cualquier cambio interno obliga a modificar la interfaz visual.

Solución: ControladorPartido actúa como facade. Expone métodos simples (crearPartido(), simularPartido(), obtenerResultado()) y oculta la complejidad del subsistema.

Justificación concreta: la UI solo interactúa con ControladorPartido. Si el motor de simulación cambia internamente, la UI no se entera.

5.3 Strategy — Tácticas de juego

Problema: los equipos pueden usar distintas tácticas (presión, defensiva, equilibrada) y cada táctica afecta diferente la probabilidad de gol. Sin Strategy, el motor de simulación necesitaría un if/switch por cada táctica posible.

Solución: la interfaz ITactica define calcularProbabilidadGol(). Cada clase concreta implementa su propia lógica. El motor trabaja contra la interfaz, no contra implementaciones.

Justificación concreta: agregar una nueva táctica solo requiere una nueva clase. No hay que tocar el motor.

5.4 Observer — Actualización de marcador y estadísticas

Problema: cada vez que ocurre un evento (gol, falta, fin de tiempo), varios módulos deben actualizarse: el marcador, las estadísticas y el relato deportivo. Si el motor llama directamente a cada módulo, el acoplamiento crece con cada nuevo componente.

Solución: el motor notifica a todos los observadores registrados a través de la interfaz IObservador. Cada observador decide cómo reaccionar al evento.

Justificación concreta: agregar un nuevo módulo de salida (por ejemplo, un log de eventos) no requiere modificar el motor, solo registrar un nuevo observador.

5.5 State — Ciclo de vida del partido

Problema: un partido tiene cuatro etapas bien definidas (primer tiempo, descanso, segundo tiempo, finalizado) y lo que se puede hacer en cada etapa varía. Sin State, el código queda lleno de if (etapa == X) dispersos por todas las clases.

Solución: cada etapa es una clase que implementa IEstado. El partido delega el comportamiento al estado actual. Transicionar de etapa es simplemente cambiar la referencia al estado siguiente.

Justificación concreta: si en el futuro se agrega tiempo extra, solo hay que crear un nuevo estado sin tocar la lógica existente.

5. Principios SOLID Aplicados

Principio	Aplicación en el sistema
S, Single Responsibility	Cada clase tiene una única responsabilidad. PartidoBuilder sólo construye partidos. Marcador solo lleva el marcador. MotorSimulacion sólo ejecuta la lógica de eventos.
O, Open/Closed	ITactica, IObservadorPartido e IEstadoPartido están abiertos a extensión (nuevas tácticas, observadores o estados) sin modificar el código existente.
D, Dependency Inversion	MotorSimulacion depende de ITactica, no de TacticaOfensiva ni TacticaDefensiva. ControladorPartido depende de IBuilder. Las clases concretas se inyectan en tiempo de construcción.

6. Patrones GRASP Aplicados

Patrón GRASP	Clase / Decisión	Justificación
Experto en Información	Partido calcula su propio resultado	Partido tiene toda la información necesaria: goles de cada equipo. Es quien debe calcular el resultado, no una clase externa.
Creador	PartidoBuilder crea instancias de Partido	PartidoBuilder agrega y configura todos los componentes de un Partido, por lo que es el responsable natural de crearlo.
Bajo Acoplamiento	MotorSimulacion - ITactica / IObservador	El motor no conoce las implementaciones concretas de tácticas ni de observadores. Esto permite cambiar o extender cualquiera sin afectar al motor.

7. Estructura de Paquetes del Proyecto Java

Paquete	Responsabilidad	Clases principales
modelo	Entidades del dominio	Partido, Equipo, Jugador, EventoDeportivo, Estadio, EstadoJugador, TipoEvento
táctica	Patrón Strategy	ITactica, TacticaOfensiva, TacticaDefensiva, TacticaEquilibrada
estado	Patrón State	IEstadoPartido, EstadoPrimerTiempo, EstadoEntretiempo, EstadoSegundoTiempo, EstadoFinalizado
observador	Patrón Observer	IObservadorPartido, Marcador, Estadisticas, RelatoDeportivo
simulación	Motor de eventos	MotorSimulacion
builder	Patrón Builder	IBuilder, PartidoBuilder
fachada	Patrón Facade	ControladorPartido
persistencia	Acceso a datos (DAOs)	ConexionDB, PartidoDATA, EquipoDATA, JugadorDATA, EstadioDATA
ui	Interfaz visual (A definir con qué tecnología)	MainApp, ControladorMenuPrincipal, ControladorConfigurarPartido, ControladorSimulacion, ControladorGestionEquipos, ControladorHistorial

8. Primera Versión del Código

El repositorio Git contiene la primera versión del código con todas las interfaces, clases abstractas y clases concretas principales. La estructura refleja exactamente la distribución de paquetes definida en el punto anterior.

Estado actual del código

Componente	Estado
Interfaces principales (ITactica, IObservador, IEstado, IPartidoBuilder)	Implementadas (Posibles modificaciones)

Clases del modelo (Partido, Equipo, Jugador, Evento)	Implementadas (Posibles modificaciones)
Patrón Builder (PartidoBuilder)	Implementado (Posibles modificaciones)
Patrón State (4 estados del partido)	Implementado (Posibles modificaciones)
Patrón Strategy (3 tácticas)	Implementado (Posibles modificaciones)
Patrón Observer (Marcador, Estadísticas, RelatoDeportivo)	Implementado (Posibles modificaciones)
Patrón Facade (ControladorPartido)	Implementado (Posibles modificaciones)
DAOs en memoria (PartidoDATA, EquipoDATA, JugadorDATA, EstadioDATA)	Implementados (Posibles modificaciones)
Persistencia SQLite con JDBC	En progreso
Interfaz visual JavaFX	En progreso

9. Informe de Avance Individual

A continuación cada integrante describe brevemente su participación hasta esta instancia.

Integrante 1 — Agustin Perez Leal

Encargado de agregar en el proyecto las primeras clases dentro de sus respectivos paquetes. Así como también de definir el alcance del proyecto junto con el integrante Pedro Smith. También se encargó de realizar parte de las modificaciones al código principal.

Integrante 2 — Franco Mioni

Encargado de corregir los bugs existentes luego de la primera implementación de código, así como también de agregar funcionalidades y necesidades faltantes en el apartado de conexión con base de datos.

Integrante 3 — Facundo Martinez Zorzi

Encargado de implementar el primer código de las clases previamente creadas por el integrante Agustin, respetando los patrones establecidos (Posibles cambios a futuros).

Integrante 4 — Pedro Smith

Encargado de definir los patrones, alcance y clases a implementar en el sistema del proyecto junto con el integrante Agustin.